

A Simplified Layout Algebra for GPU Kernels

Nan*

Draft of September 7, 2026

Abstract

The layout algebra behind NVIDIA’s CuTe library describes how tensor tiles map onto threads and memory. A layout is a shape with a stride for each axis, so the address of a coordinate is the sum of its indices times their strides. As formalized by Shah and by Cecka, its central operations, such as composition, complement, division, product, are guarded by at least six interacting admissibility conditions. We show that this complexity is an artifact of a representational choice, not of the domain.

We present a layout algebra in which every layout names the space its inputs and outputs live in: logical indices, physical indices, or thread-value indices. A layout into logical space must be a bijection from its N coordinates onto $[0, N)$. A layout into physical space must be alias-free: different coordinates land on different addresses. The exception is broadcast, where many threads read one value on purpose, and the layout has to mark where that happens. Nothing maps out of a physical index.

Composing f with g requires only that f ’s range be exactly g ’s flattened set of coordinates. To emit the index arithmetic, naively, we can always compute f , split that by division and remainder into g ’s coordinates, and feed it into g . Often the composite can be simplified to a layout as well. We show that if it can be written this way the form is unique, and we emit that simplified form.

A warp’s or a thread’s slice is a *restriction*. A restricted layout is no longer composable, which forces the user to settle the whole view of a tile before taking slices.

We validate the algebra by showing that CuTe’s operations, such as composition, complement, and division, can be reproduced in it. Furthermore, we provide an implementation in OCaml, and use it to rederive the atoms and usage patterns of CuTe.

The implementation is roughly 900 lines of code.

1 Introduction

For GPU kernels, it is important to have a scheme for describing how to move tiles of tensors between global memory, shared memory and registers. The de facto vocabulary for such schemes is the CuTe layout algebra underlying NVIDIA’s CUTLASS library [1, 2]. The most fundamental concept is the layout, which we shall reuse heavily.

Definition 1 (Layout). A *shape* is a positive integer n , or a tuple $(S_1, \dots, S_k)$ of shapes; its *size* is $\text{size } n = n$ and $\text{size}(S_1, \dots, S_k) = \prod_i \text{size } S_i$. A *coordinate* of S is an integer $i \in [0, n)$, respectively a tuple $(c_1, \dots, c_k)$ of coordinates of the S_i ; write $\text{Coord}(S)$ for the set of them. A *layout* $S:D$ is a shape S together with a *stride* D of the same structure: an integer where

*nan@silares.com. TODO: full name and affiliation.

S is a positive integer, and a tuple $(D_1, \dots, D_k)$ where S is $(S_1, \dots, S_k)$. Its index function $\text{Coord}(S) \rightarrow \mathbb{N}$ is defined by recursion on the shape,

$$(n:d)(i) = i d, \quad ((S_1, \dots, S_k):(D_1, \dots, D_k))(c_1, \dots, c_k) = \sum_{j=1}^k (S_j:D_j)(c_j).$$

Flattening is a bijection $\langle \cdot \rangle_S : \text{Coord}(S) \rightarrow [0, \text{size}S)$, taken row-major, so that the last component varies fastest, with inverse unflatten_S . A coordinate may therefore be given as a single integer, and we say that integer *is* the coordinate. The *cosize* of the layout is the least N with its image inside $[0, N)$, which is one plus the largest value it takes. The layout is *dense* when its image is an initial segment $[0, N)$ of the integers, and a *dense bijection* when it is moreover injective, in which case $N = \text{size}S$ and every value below N is taken exactly once. A *mode* is one entry of the shape together with the matching entry of the stride. Two layouts *concatenate*: for $A = S:D$ and $B = S':D'$, the layout (A, B) is $(S, S'):(D, D')$, and it sends (c, c') to $A(c) + B(c')$.

Layouts are fundamentally the correct concept here, because physical memory is linearly addressed while the logical space is multidimensional and dense as we imagine it. They are a natural way to abstract and describe those mappings.

The algebra has four operations. Three of them, composition, complement and logical division, are formalized by Shah [3]; the fourth, logical product, we take from the CUTLASS implementation. The *complement* of a layout $B = (N_0, \dots, N_\alpha):(d_0, \dots, d_\alpha)$, sorted by stride, with respect to an integer M , is

$$B_M^* = \left(d_0, \frac{d_1}{N_0 d_0}, \dots, \frac{M}{N_\alpha d_\alpha} \right) : \left(1, N_0 d_0, \dots, N_\alpha d_\alpha \right),$$

chosen so that the concatenation (B, B_M^*) has size M and its layout function is a bijection of $[0, M)$ onto itself. *Logical division* is $A / B = A \circ (B, B_{\text{size}A}^*)$, which reindexes A so that one mode runs inside the tile B selects and the other runs across copies of it. *Logical product* is $A \otimes B = (A, A_M^* \circ B)$, where $M = \text{size}A \cdot \text{cosize}B$, the smallest M for which the composition is defined. Its first mode is one copy of A , and its second walks B through the addresses A does not occupy, so the result is $\text{size}B$ copies of A placed in the pattern B names.

These operations are critical for constructing the layouts one usually wants. There is, however, a lot of complexity underneath them. Layouts are in general non-injective and non-surjective onto $[0, M)$, and CuTe always wants to write the composite as a layout again. This causes at least six condition checks. Two are in the complement, whose shape entries are quotients and so must divide: $N_{i-1}d_{i-1} \mid d_i$ for every i , and $N_\alpha d_\alpha \mid M$. The others are in composition. Write the flattened shape of A as $M_0 \cdots M_\alpha$. It is *left divisible* by r when $r = M_0 \cdots M_{i-1} \cdot c$ for some i , where c is smaller than M_i and divides it, and *weakly left divisible* when the same holds except that c need not divide M_i . For each mode $(N):(r)$ of B this is three checks, since the prefix must divide r , the leftover c must divide M_i , and what remains after r must be weakly left divisible by N . One more is required across modes, that the intervals on which they act be pairwise disjoint [3]. None of these conditions is visible in the CuTe API, yet all of them are visible failures.

We try to resolve those complexities by restricting what a layout can be.

Explicit index spaces. The input and the output of a layout each represent one of three things: a logical index into a tile, a thread-value index, or a physical offset in memory. Therefore

a layout should declare which of the three its input and its output are, and we allow only the maps worth having between them. Logical and thread-value indices map into one another in either direction. Logical indices map into physical offsets. Nothing maps out of a physical offset. By design, a thread reaches memory by composing its map into logical space with the storage map.

Requirements that follow from the space. We allow those maps and not others because the three spaces do not carry the same obligations. Logical and thread-value indices have to account for their contents losslessly, so a map into either must be a dense bijection. Composition can only happen when one layout’s image and the other’s domain agree completely, which is natural because a physical offset is forbidden as a domain. A physical offset carries far fewer requirements, since gaps in memory are common and sharing one address between threads, to read the same element, is often what is wanted, so a map into physical need only be alias-free, meaning that distinct coordinates reach distinct addresses except where the layout marks a broadcast.

Composition and simplification. The composite of two layouts is not a layout in general. But we can decide if it can be written as a layout, and when it is a layout we emit one weighted sum of the coordinates. When it is not, we emit the composition literally, at the cost of a longer expression and more work at run time.

2 Our construction of layouts

We build on top of Definition 1. First we define the spaces.

Definition 2 (Spaces). Each side of a layout carries a space from $\{\text{logical, physical, thread-value}\}$, written $A \rightarrow B$. A layout may be annotated $A \rightarrow B$ for $A, B \in \{\text{logical, thread-value}\}$, or $\text{logical} \rightarrow \text{physical}$, and no other pair is constructible. Composition takes $A \rightarrow \text{logical}$ and $\text{logical} \rightarrow C$ to $A \rightarrow C$, so $\text{thread-value} \rightarrow \text{physical}$ exists only as a composite.

Definition 3 (Replication). Extend Definition 1 by allowing the stride against a positive integer n to be the mark $\perp$ in place of an integer, with

$$(n:\perp)(i) = 0 \quad \text{for every } i \in [0, n).$$

We denote $\text{Axis}(n, s)$ for $n:s$ and $\text{Bcast}(n)$ for $n:\perp$.

A zero stride is rejected rather than read as replication. This is because replication is legal for reads and not for writes, since simultaneous loads of one address are a broadcast while simultaneous stores are a race. It is better not to use 0, so that the two can never be mixed.

Definition 4 (Swizzled layout). A *swizzle* is a bijection $\text{physical} \rightarrow \text{physical}$ of the form

$$\sigma(x) = x \oplus (((x \gg \text{src}) \wedge (2^b - 1)) \ll \text{dst}),$$

its two fields $[\text{src}, \text{src}+b)$ and $[\text{dst}, \text{dst}+b)$ disjoint. A layout $A \rightarrow \text{physical}$ followed by a swizzle is a *swizzled layout*, again $A \rightarrow \text{physical}$.

A swizzle runs from physical to physical, so it can only ever be the last step, and being a bijection it sends distinct addresses to distinct addresses. It therefore leaves the conditions below untouched, and matters only in the arithmetic that is finally emitted.

Definition 5 (Composition). Let $f : A \rightarrow \text{logical}$ and let $g : \text{logical} \rightarrow C$ have domain shape S_g . The *composite* $g \circ f : A \rightarrow C$ is defined when f is a dense bijection onto $[0, \text{size} S_g)$, and is the map

$$g \circ f = c \mapsto g(\text{unflatten}_{S_g}(f(c))),$$

whose domain shape is that of f .

Definition 6 (Declared replicas). For a layout L with domain shape S , define a relation $\sim_L$ on $\text{Coord}(S)$:

- if L is a layout, $c \sim_L c'$ when c and c' agree at every position whose stride is an integer, that is, they differ only at positions marked $\perp$;
- if $L = g \circ f$ with g of domain shape S_g , then $c \sim_L c'$ when $\text{unflatten}_{S_g}(f(c)) \sim_g \text{unflatten}_{S_g}(f(c'))$.

Definition 7 (Validity). Let $L : A \rightarrow B$ be a layout with domain shape S . Then L is *valid* when

$$\begin{cases} L \text{ is a dense bijection} & \text{if } B \in \{\text{logical}, \text{thread-value}\}, \\ L(c) = L(c') \implies c \sim_L c' & \text{if } B = \text{physical}. \end{cases}$$

It is *write-valid* when $B = \text{physical}$ and in addition

$$L(c) = L(c') \implies c = c'.$$

Only a write-valid layout may be the target of a store.

Whether a layout is a dense bijection can be decided without enumeration.

Lemma 1 (Dense bijections). *Let L be a layout with no marked positions, its positions carrying sizes $n_1, \dots, n_k$ and strides $s_1, \dots, s_k$ with every $n_i \geq 2$, and put $N = \prod_i n_i$. Then L is a dense bijection onto $[0, N)$ if and only if, after sorting the positions by stride, $s_{(1)} = 1$ and $s_{(j+1)} = s_{(j)} n_{(j)}$ for every j : the sorted strides are the running products of the sorted sizes.*

Proof. ($\Leftarrow$) With running-product strides the map is exactly mixed-radix representation with radices $n_{(1)}, \dots, n_{(k)}$, a bijection onto $[0, N)$. Positions of size 1 contribute nothing and are dropped beforehand.

($\Rightarrow$) Sort the positions by stride and write $N_j = \prod_{i \leq j} n_{(i)}$ (so $N_0 = 1$). *Claim:* $s_{(j)} \leq N_{j-1}$ for every j . Every image value below $s_{(j)}$ comes from a coordinate supported on positions $1, \dots, j-1$, since a nonzero index at any later position contributes at least $s_{(j)}$ and no contribution is negative; those positions produce at most N_{j-1} distinct values, so surjectivity onto $[0, s_{(j)})$ forces $s_{(j)} \leq N_{j-1}$. Consequently

$$\sum_j (n_{(j)} - 1) s_{(j)} \leq \sum_j (n_{(j)} - 1) N_{j-1} = N - 1,$$

the last equality by telescoping. But the left side is the maximum of the image, and surjectivity onto $[0, N)$ requires the maximum to be exactly $N - 1$. The two sums therefore agree, and since each term on the left is at most the corresponding term on the right, they agree termwise; as $n_{(j)} - 1 \geq 1$ this gives $s_{(j)} = N_{j-1}$ for every j , which is the running-product condition. $\square$

3 Operations

The operations below transform layouts, and their behaviour on a swizzle is stated where it matters. An *arrangement* is a layout required to be a dense bijection, checked by Lemma 1 at the call; its strides carry order and nothing else. An arrangement cannot express padding, because a gap in it would be a whole footprint wide. Padding lives in the strides of the tile being copied, and the tile’s cosize then spaces the copies.

`canonical( $S$ )` the row-major layout of S : last axis stride 1, running products leftward. Dense by Lemma 1.

`split( $by:S$ )` regroups one position $\text{Axis}(n, s)$ with $\text{size } S = n$ into the structure of S , giving each new position the stride s times its running product. The index i is read as the S -coordinate of i , so the map is unchanged. A marked position splits into marked positions.

`divide( $by:S$ )` the *partition* (CuTe’s `zipped_divide` with a shape tiler). Per axis, size n_i splits into $(n_i/k_i, k_i)$ (divisibility required); the pieces regroup as $\text{Group}[\text{tile axes}; \text{rest axes}]$ with coordinate $((\text{tile}), (\text{which tile}))$. It is the relabeling $\rho((t_i), (b_i)) = (b_i k_i + t_i)_i$, a bijection, with $\text{divide } t = t \circ \rho$. Every tile is kept, so the rest needs no complement operation. Selecting one tile is restriction (Section 5), afterwards.

`repeat( $by$ )` copies side by side: $\text{Group}[by \times \text{cosize } t; t]$, the arrangement’s strides scaled to footprint units. A dense arrangement over a dense tile is dense, since $\text{cosize } t = \text{size } t$ there and the two digit systems concatenate. On a swizzled layout, repeat is accepted iff both swizzle fields lie below the (power-of-two) footprint: multiples of the footprint then commute with the swizzle, so each copy is independently swizzled (the multi-stage shared-memory buffer); otherwise it rejects—the copies would contaminate one another’s swizzle.

`interleave( $by$ )` copies dealt element-wise (CuTe’s raked product), the mirror image: $\text{Group}[by; t \times \text{size}(by)]$. Blocked versus raked is the coalescing distinction: with threads as the arrangement, interleave gives consecutive lanes consecutive addresses. No meaning is defined for interleaving a swizzled layout (no kernel pattern rakes swizzled tiles); it rejects.

`broadcast( $by:S$ )` copies at the *same* addresses: $\text{Group}[\text{Bcast-tree of } S; t]$. The dual of repeat; the arrangement is a shape because aliased copies have no positions to give. Read-legal, write-illegal by the Bcast rule.

`coalesce` flattens nesting, drops positions of size 1, merges $\text{Axis}(a, s')$ into an inner neighbour $\text{Axis}(b, s)$ when $s' = s b$, and merges adjacent marked positions. The map is unchanged. It is an optional normal form here, never a precondition of anything.

`inverse` of a dense bijection, defined on layouts: the index j maps to the coordinate sent to j , whose position i carries index $(j/s_i) \bmod n_i$. The spaces swap, so the inverse of a thread-value map $\text{thread-value} \rightarrow \text{logical}$ is the *ownership map* $\text{logical} \rightarrow \text{thread-value}$, which element is held by which thread and value. Its use is *retiling*: two thread-partitionings of one tile, a copy’s fragment and an MMA’s operand, are related by $\text{inverse}(\text{mma}_{TV}) \circ \text{copy}_{TV}$, carrying a copy coordinate to an element to an MMA coordinate. That correspondence is a bijection and need not fix the lane: with 32 lanes holding two values each, $\text{copy} = 2 \text{ lane} + v$ against $\text{target} = \text{lane} + 32 v$ sends 62 of the 64 coordinates to a different lane. A composite is a bijection too, so its inverse function exists, but that function need not be a layout. The

layout $(3, 2):(1, 3)$ sends the six indices to 0, 3, 1, 4, 2, 5, and composed with itself it sends them to 0, 4, 3, 2, 1, 5: that map is its own inverse and is a layout over no refinement of six. An inverse may instead be kept as a composite, with the factors inverted in reverse order. That needs every factor invertible, so it applies only to chains that stay left of physical; storage is alias-free rather than injective and may leave gaps, so a composite ending in memory has no inverse at all.

Proposition 1 (Closure). *split, divide, and coalesce preserve the function up to the stated relabeling, hence preserve density and injectivity. repeat and interleave with dense arguments preserve density; with an injective tile they preserve injectivity. broadcast preserves the image and multiplies multiplicity uniformly.*

Proof. Split and coalesce regroup digits in place, leaving the index unchanged, and divide composes per-axis splits with the permutation ρ . For repeat the images are $\{a \cdot \text{cosize}t + v : a \in \text{Im}(by), v \in \text{Im}(t)\}$; density of by makes a range over $[0, \text{size}by)$ and, for dense t , $\text{cosize}t = \text{size}t$, so the two digit systems concatenate. Injectivity survives a tile with gaps because distinct copies are separated by at least the footprint. interleave is symmetric. $\square$

4 Composition

Composition is the only place two layouts meet, and the type does most of the work:

$$\text{compose} : (\alpha, \text{logical}) \text{ Layout} \rightarrow (\text{logical}, \gamma) \text{ Layout} \rightarrow (\alpha, \gamma) \text{ Layout}.$$

Physical cannot be an intermediate because no signature accepts it on the left, and two thread-value maps cannot be composed with each other because they share a codomain, not an interface. The composition chain is therefore short and terminal: (thread-value $\rightarrow$ logical) maps compose through zero or more (logical $\rightarrow$ logical) relabelings into one (logical $\rightarrow$ physical) storage map, after which nothing composes.

The condition on f is decided by Lemma 1 when f is a layout and by enumeration when it is a composite, which happens in chains. It rejects a marked position on the left, since replication is not injective. Nothing else is checked.

CuTe’s `composition` of two layouts must return a layout, so it has to align the operands’ digit structures and succeeds only when their shapes and strides divide one another suitably. It carries a second representation, `ComposedLayout`, for the cases where that fails, and that representation has a reduced contract: no `stride()`, alignment queries pinned to 1, a zero-offset requirement when it is the right operand of a composition with an ordinary layout, and structural operations delegated to one factor. Two separable things are at work here. The spaces and their requirements decide which maps may be built at all, which is what CuTe leaves to convention. Keeping the operand pair is what removes the divisibility conditions, and `ComposedLayout` shows that CuTe can keep a pair too. The combination is what differs: one representation with one contract, whether or not the operands would have fused, and the arithmetic recovered afterwards by Theorem 1, exactly when it exists.

Example 1 (A composite that is not a layout). Let f be the relabel $(a, b) \mapsto 3a + b$ of shape $(2, 3)$ and let g be the storage $(3, 2):(1, 3)$. Both are valid, and the composite sends the size-3 axis at $a = 0$ to the addresses 0, 3, 1. No layout has this map, since along one axis a layout runs in an arithmetic progression and a prime-sized axis cannot be split, so a composition that must

return a layout has to reject this pair. Here the composite exists and is a bijection onto $[0, 6)$, emitted as $\lfloor x/2 \rfloor + 3(x \bmod 2)$ for $x = 3a+b$, and Theorem 1 proves that no layout describes it.

4.1 Emitting a composite

The emitter writes Definition 5 out as an integer expression over the composite’s coordinate variables:

$$E ::= k \mid c \mid \sum_i a_i E_i + k \mid \lfloor E/w \rfloor \mid E \bmod r \mid E \oplus E \mid \text{let } x = E \text{ in } E,$$

with k, a_i, w, r compile-time constants and each coordinate variable c ranging over $[0, n_c)$, the size of its position. A layout with strides s_i writes $\sum_i s_i c_i$, and a marked position contributes nothing. A composite binds the index f computes and reads g ’s digits off it: digit j , of radix n_j and weight w_j (the product of the radices to its right), is $\lfloor x/w_j \rfloor \bmod n_j$, without the modulus on the leading digit since composability gives $x < \text{size}(g)$. A swizzle likewise binds the address it is given and exclusive-ors one shifted field of it. Binding is what keeps this small: a stage index is computed once however many digits are taken from it. This step cannot fail; it is the pair, written out. For the running example—the thread-value map $(cg, (r, v)) \mapsto 4cg + 16r + v$ onto column-major (8, 16) storage—it writes

$$\text{let } x1 = 4*c0 + 16*c1.0 + c1.1 \text{ in } (x1 / 16) + 8*(x1 \% 16).$$

When the composite is a layout, its address is a single weighted sum, and this expression hides that. Recovering it looks like a simplification problem, and we first treated it as one, with a bounds-aware rewrite system of the kind Halide [9], TVM [10] and MLIR’s affine dialect [11] carry. That was the wrong tool. It missed layouts that exist, and where none exists it made the code worse, since exposing cancellations means inlining the bound index into every digit and splitting the digits apart: on random operands that inflated the arithmetic by 45% over the expression above, in 400 of 628 cases. The next subsection decides the question instead.

4.2 Deciding whether a composite is a layout

A composite is a function on a finite box, and at compile time every one of its values is available. Whether it is a layout, over a possibly finer shape, is therefore decidable by evaluation rather than by searching for a representation. The procedure below costs $O(\text{size})$ evaluations, which is what the checks of Section 7 already spend, plus work linear in each axis.

Fix a map $f : B \rightarrow \mathbb{N}$ on the box $B = \prod_{i=1}^m [0, n_i)$, and write e_i for the unit coordinate on axis i and $\mathbf{0}$ for the origin.

Definition 8 (Refinement; layout over a refinement). A *refinement* of axis i is a strictly increasing chain of divisors $1 = w_1 \mid w_2 \mid \dots \mid w_k \mid n_i$ with $w_k < n_i$, presenting c_i by its digits $d_j(c_i) = \lfloor c_i/w_j \rfloor \bmod r_j$ with $r_j = w_{j+1}/w_j$ and $w_{k+1} = n_i$. Starting at 1 and ending below n_i is what makes the digits *cover* the axis: every radix is at least 2 and $\prod_j r_j = n_i$, so $(d_1, \dots, d_k)$ is a mixed-radix presentation of c_i . For $n_i = 1$ the chain is empty and there are no digits. A *layout over the refinement* is $f(c) = k + \sum_{i,j} s_{ij} d_j(c_i)$ with integer k and s_{ij} , where a stride may be zero. The trivial refinement, one digit of radix n_i per axis, gives the forms that are affine in the coordinates themselves.

Being a layout over a refinement of the domain shape is what CuTe’s composition must produce, and what its admissibility conditions are conditions for.

Lemma 2 (Separability). *f is a layout over some refinement if and only if*

$$f(c) = f(\mathbf{0}) + \sum_{i=1}^m g_i(c_i), \quad g_i(v) = f(v e_i) - f(\mathbf{0}), \quad (1)$$

and each g_i is a layout over some refinement of $[0, n_i]$.

Proof. ($\Rightarrow$) Every digit is a function of one coordinate, so such a layout is $f(c) = k + \sum_i h_i(c_i)$ with $h_i = \sum_j s_{ij} d_j$. Evaluating at $\mathbf{0}$ and at $v e_i$ gives $h_i(v) - h_i(0) = g_i(v)$, so g_i is h_i shifted by a constant—a layout over the same refinement—and (1) holds. ($\Leftarrow$) Sum the per-axis forms. $\square$

Separability is one pass over the box. It leaves m independent one-dimensional problems, and each is settled in time linear in its axis.

Lemma 3 (Floor form). *For $w \mid w'$, $\lfloor v/w \rfloor \bmod (w'/w) = \lfloor v/w \rfloor - (w'/w) \lfloor v/w' \rfloor$. Hence g is a layout over a chain $w_1 \mid \cdots \mid w_k \mid n$ if and only if $g(v) = \sum_j a_j \lfloor v/w_j \rfloor$ for integers a_j on the same chain, and the two coefficient families determine each other by $a_j = s_j - r_{j-1} s_{j-1}$ and $s_j = g(w_j)$.*

Proof. The identity is the definition of the remainder, valid because $w \mid w'$. Substituting it into $\sum_j s_j d_j$ and collecting the terms in $\lfloor v/w_j \rfloor$ gives $a_j = s_j - r_{j-1} s_{j-1}$ with $s_0 = 0$; the relation is triangular with unit diagonal, hence invertible over $\mathbb{Z}$. Evaluating $\sum_i a_i \lfloor w_j/w_i \rfloor$ at $v = w_j$ kills every term with $w_i > w_j$ and leaves $\sum_{i \leq j} a_i (w_j/w_i)$, which the same recurrence identifies with s_j . $\square$

Theorem 1 (Linear recognition). *Let $g : [0, n] \rightarrow \mathbb{Z}$ with $g(0) = 0$ and let $h(t) = g(t) - g(t-1)$ for $1 \leq t < n$. If $n = 1$, return the empty refinement. Otherwise begin the refinement with $w_1 = 1$, set $w_{\text{last}} = 1$, and process $t = 1, 2, \dots, n-1$ in order: whenever $h(t) \neq 0$, reject unless $t \mid n$ and $w_{\text{last}} \mid t$; otherwise record the coefficient $a_t = h(t)$, append t to the refinement if $t > 1$, set $w_{\text{last}} = t$, and subtract a_t from h at every multiple of t . Then g is a layout over some refinement if and only if the scan completes without rejecting; the reported weights are necessary boundaries of any such refinement, so together with $w_1 = 1$ they are the weights of the coarsest one; the strides are $s_j = g(w_j)$; and the whole scan runs in $O(n)$ time.*

Proof. By Lemma 3 such a layout is equivalent to $g = \sum_j a_j \lfloor \cdot / w_j \rfloor$ over a divisor chain ending at n . First differences of $\lfloor v/w \rfloor$ are the indicator of the multiples of w , so $h(t) = \sum_j a_j \lfloor w_j \mid t \rfloor$. Induct on t : when the scan reaches t , the contributions of all weights below t have been subtracted, so the residual $h(t)$ equals a_t , the coefficient at weight t , which is therefore forced—and nonzero exactly when t is a boundary. Necessity of the divisibility tests is immediate: the weights of a refinement divide n and form a chain. Conversely, if the scan completes, the reported weights are a chain of divisors of n and the recovered a_j reproduce g by construction, so Lemma 3 converts them to strides $s_j = g(w_j)$. Since each a_t is forced, a boundary with $a_t \neq 0$ appears in every refinement admitting a form, which is the claimed necessity. For the running time, each accepted weight is a strictly larger multiple of its predecessor, so accepted weights at least double; the subtraction loops therefore perform fewer than $n + n/2 + n/4 + \cdots < 2n$ updates, alongside the single scan. Finally, $w_1 = 1$ belongs to the reported refinement whatever a_1 is, because every refinement carries it (Definition 8); a zero coefficient there means a zero stride, not an absent digit. For $g(v) = \lfloor v/2 \rfloor$ on $n = 4$ the differences are 0, 1, 0 and only weight 2 is reported, but the refinement is (2, 2) with strides (0, 1), and omitting its first digit would leave radices whose product is not n . A constant g —necessarily 0, since $g(0) = 0$ —gives the trivial refinement at stride 0. $\square$

The characterization these two lemmas state—digits as differences of floor terms, first differences as indicators of divisibility, and the coefficients they force—is also the machinery of Appendix A.2.1 of Axe [7], whose Lemmas A.3–A.6 and Theorem A.8 use it to prove a normalized sharded representation unique by reconstructing it from the function it induces. We claim no novelty for it. What Theorem 1 adds is the question asked of it: the input is an arbitrary map on a finite box rather than one assumed representable, so existence is *decided* and non-representable maps are rejected; the extraction is an incremental sieve rather than Möbius inversion over the divisor lattice, which is what gives the linear bound; and a rejection is not a dead end, because the emitter has the pipeline of Section 4.1 to fall back on.

No factorization and no search over orderings appears anywhere, since the boundaries announce themselves. Example 1 is settled by the scan rather than argued: its size-3 axis has differences 3, -2 , forcing a weight at 1 and again at 2, and $2 \nmid 3$. Because the procedure reads values, a swizzle is no obstacle to it; the layout $2c$ under $\{1; 1; 0\}$ emits as $(2c) \oplus (\lfloor 2c/2 \rfloor \bmod 2)$ and is decided to be $3c$. A restriction is decided the same way over its free coordinates, with the pinned contribution folded into the constant.

The emitter therefore has two outputs. If the map is a layout over some refinement it emits that layout’s arithmetic, over the coarsest refinement and with the coefficients the map forces, so no form with fewer digits exists. Otherwise it emits the expression of Section 4.1.

5 Restriction

A kernel does not use a whole address map. Warp w and thread t each use a slice of one. Fixing a coordinate cannot produce a layout, since the fixed contribution is a position rather than a pattern, and it cannot be handed back as a separate offset either, because $\sigma(64 + x) \neq 64 + \sigma(x)$ whenever the swizzle reads the fixed bits. So the fixed coordinates are kept as data beside the layout.

Definition 9 (Partial coordinate; restricted layout). A *partial coordinate* of shape S is $\pi ::= \text{Free} \mid \text{At } i \mid \text{Parts}[\pi_1, \dots, \pi_k]$, structurally matching S with in-bounds indices. A *restricted layout* is a pair (L, π) of an *unmodified* layout and a partial coordinate of its domain shape. Its shape replaces fixed slots by B 1; its offset function *completes* the coordinate (fixed slots take index 0 and contribute their At index) and evaluates L .

Nothing is folded or split, so restriction applies to every map, layout or composite, swizzled or not, with no case analysis. Only a physical codomain may be restricted: choosing which addresses this executor touches is an execution decision, whereas density in logical space forbids discarding elements. Slicing therefore happens after composition, where the warp and thread coordinates are slots of the composite’s domain. Restricting again takes the union of the two partial coordinates, so slicing by warp and then by thread is associative.

Because layouts and restrictions are closed data, a slice needs no separate treatment: the restricted map is decided in its own right over the free coordinates, with the pinned contribution folded into the constant, and falls back to the pipeline with those variables replaced by their constants. Warp 2’s slice of the composite above emits $r + 8v + 64$. The per-lane math is generated from the layout, not tabulated.

6 CuTe’s operations in this algebra

We now express CuTe’s central operations in this algebra and execute the correspondence on NVIDIA’s own worked examples [1]. (Convention translation is mechanical: a CuTe layout’s mode list reverses, recursively, into our row-major tree; CuTe’s $\text{composition}(A, B)$ applies B first and is our compose with $f = B, g = A$.)

Composition. All three documented examples are reproduced as functions—our composite agrees pointwise with the strided layout CuTe prints, including the mode-splitting example $\text{composition}((6, 2):(8, 2), (4, 3):(3, 1)) = ((2, 2), 3):((24, 2), 8)$. Every published example has a dense right operand, so all of them compose here.

Complement. There is no complement operator here. divide carries the rest by construction, and the density check decides complement’s defining property. Both directions of $\text{complement}(4:1, 24) = 6:4$ are witnessed by the single divide of $\text{canonical}(24)$ by $B\ 4$, since tile and rest are each other’s complements. The strided case $\text{complement}(4:2, 24) = (2, 3):(1, 8)$ comes from dividing the $(12, 2)$ reshape. All six documented complements are checked to make (B, B^*) dense onto $[0, 24)$.

Division. CuTe’s strided-tiler example $\text{logical_divide}((4, 2, 3):(2, 1, 8), 4:2)$ equals our composition with the completed tiler $(B, \text{complement}(B, 24))$ on the left, which is their division carried out as a composition with a dense bijection, on their numbers.

Product, and one divergence. Their logical_product example is verified against their formula. On a tile with gaps the two differ: CuTe’s product places copies into the tile’s gaps, while repeat keeps them, since padding is a property of the storage and no operation here repurposes it.

Which conditions go, and which stay. Two kinds of condition have to be told apart. The divisibility conditions exist so that composition and complement can return a layout: left divisibility, weak left divisibility, disjoint intervals of definition, complement’s two divisibility requirements, and division’s lemma that the pair it assembles is admissible in turn. Nothing here fuses, so all of those go, and complement goes with them because division carries its rest.

What stays is not empty, and none of it is about representation. Composition requires the left operand to cover the right operand’s domain exactly, which is stronger than the containment ordinary function composition needs, and is decided for layouts by Lemma 1. A map into logical or thread-value must be a dense bijection, a map into physical must be alias-free, and a store target must be injective. Three operations carry requirements of their own: divide needs each axis size divisible by its tile size, repeat on a swizzled layout needs both swizzle fields below the footprint, and interleave rejects a swizzled layout. These are obligations about what a map means, not about whether two maps can be written as one.

7 Coverage of CuTe usage

The previous section covered CuTe’s operators. This one covers the patterns CUTLASS kernels are written with, and how each is expressed here. The last column names the suite that executes

the expression (Section 8); a dash means it is expressible but no test exercises it.

CuTe idiom	Here	Verified
<code>make_layout(shape, stride)</code>	a layout	throughout
<code>composition(A, B)</code>	<code>compose(B, A)</code> (kept as the pair)	Suite 1, all three documented examples
<code>complement(B, M)</code>	the rest component of divide; its defining property decided by the density check	Suite 1, all six documented examples
<code>logical_divide, zipped_divide</code>	divide: $((\text{tile}), (\text{rest}))$, every tile kept	Suite 1 (strided-tiler example via the completed tiler); Suites 2–3
<code>local_tile</code>	divide, then restrict on the composite pinning the rest coordinate to this block’s tile	Suite 2 (warp-tile derivation), Suite 3
<code>local_partition, partition_A/B/C, tensor(_, k)</code>	restrict on the composite: pin the executor’s coordinates	Suite 3 (stage slice), warp-tile derivation
<code>logical_product, blocked_product, tile_to_shape</code>	repeat with the order as a dense arrangement	Suite 2 (staged buffer; SM90 GMMA accumulator)
<code>raked_product</code>	interleave	Suite 2 (blocked vs raked), Suite 3 (copy partition)
stride-0 modes (replication)	broadcast / marked positions, read-only by rule	Suite 2 (GMMA operand replication)
<code>Swizzle<B,M,S> ◦ layout</code>	<code>with_swizzle {B; src; dst}</code> on a storage layout	Suite 2 (SW32/64/128 atoms, bank property)
multi-stage pipeline buffer	repeat of the swizzled tile (legality: fields below the footprint)	Suite 2 (3-stage SW128), Suite 3
<code>TiledCopy: partition_S/D</code> with vector width	divide by the vector, interleave over lanes (lanes fast $\Rightarrow$ coalesced)	Suite 3 (coalescing, vectorization asserted)
<code>TiledMMA: fragment over a CTA tile</code>	fragment atom interleaved by the warp grid, composed onto the tile	Suite 2 (SM90 GMMA accumulator from the SM80 <i>C</i> fragment), warp-tile derivation
<code>right_inverse, retile_S/D</code>	inverse: the ownership map; retiling is the correspondence $\text{compose}(\text{copy}_{TV}, \text{inverse}(\text{mma}_{TV}))$	<code>test_layouts</code> (inverse law, retile correspondences)
<code>coalesce</code>	coalesce, optional	<code>test_layouts</code>
<code>group_modes, select, flatten</code>	tree regrouping of Group; coalesce	—
<code>recast, upcast</code>	divide by the vector width; the unit change is outside the algebra	—
TMA box, multicast	divide by the box shape; broadcast across CTAs	— (box), Suite 2 (replication)
identity tensor + predication	masking, outside the layout algebra (Section 10)	—

The middle column uses eight operations, namely `compose`, `divide`, `restrict`, `repeat`, `interleave`, `broadcast`, `with_swizzle` and `inverse`, together with `coalesce`. No row needs an operation absent from Section 3.

Four rows are unexercised. Mode surgery and unit changes regroup a shape without touching composition or tiling. Predication is masking, which Section 10 places outside the algebra. The TMA box is divide by the box shape, so it uses nothing the verified tiling rows do not; what is untested there is the descriptor it feeds, which is not a layout question. Composition, tiling and partitioning appear only in verified rows.

8 Evaluation

The evaluation is four executable suites over an implementation of roughly 630 lines of OCaml for the algebra, 130 for the decision procedure and 150 for the expression type it emits (1,250 with interfaces and their documentation), and 1,700 lines of tests; every law stated in this paper is asserted in them.

Transcription and derivation. A layout enters a test suite one of two ways. A *transcription* copies a published layout and checks properties of the data, which tests the representation. A *derivation* builds the layout from the operations and compares it against the transcription, which tests the algebra. A missing operation is invisible to transcription, because the transcriber performs it mentally, and divide went unnoticed here for exactly that reason until a derivation needed it. Hardware facts, the PTX fragment tables and the swizzle parameters, are the only layouts we transcribe; everything built from them is derived.

Emission oracle. Every map any suite builds, 178 of them, is emitted both ways, as the expression of Section 4.1 and as the decided form, and both are evaluated at every coordinate against the map’s own evaluation. A form affine in the coordinates is fitted to each by enumeration, from the origin and the unit coordinates and then checked at every point, and the harness asserts that whenever the fit succeeds the decision procedure returns exactly that sum, never a different form and never “none”. Of the 178, 147 emit as one weighted sum, 9 as digits of a single coordinate, and 22 as the expression.

Suite 4: random layouts. Random operands of the two shapes composition sees, a dense relabel from a random factorization and a storage map of random axis order with random padding. 600 pairs, 300 three-stage chains and 300 swizzled composites, all checked at every coordinate under the same oracle. Of the pairs, 327 are affine in their coordinates and every one is decided to that sum, 524 are layouts over some refinement, and the decided forms use 686 divisions and moduli against the 1,026 of the emitted expression. Of the chains, 119 are affine and all 119 are decided, which is the case the rewrite system missed.

Suite 1: CuTe’s central algebra (Section 6): three composition examples, six complements, the strided-tiler division, the product example, and the documented divergence.

Suite 2: hardware atoms and layouts derived from them. The transcribed atoms are checked against PTX: SM70 quadpair fragments, the SM80 `mma.m16n8k16` A , B and C fragments, the `ldmatrix m8n8` fragment, and the GMMMA shared-memory swizzle atoms. The last are checked against the CUTLASS source to be the family $\{B; src=6; dst=3\}$ over $(8, 8 \cdot 2^B)$ half-elements for $B \in \{0, 1, 2, 3\}$, and the SW128 bank property, that the unit index becomes $u \oplus r$, is asserted. Two composites are then derived from those atoms rather than transcribed:

a three-stage swizzled pipeline buffer, and the SM90 GMMMA 64×128 accumulator layout, built from the SM80 `mma.m16n8k16 C` fragment by `divide(16,8)`, then interleave by the canonical `(4,16)` rest grid, then composition. It equals NVIDIA’s printed layout at all 8192 coordinates.

Suite 3: a GEMM mainloop from the operations alone. The tile is $(8, 32)$ in halves, global leading dimension 40, 16-byte vectors, one copying warp, and two shared-memory stages of the SW64 atom. The writer is `divide(1,8)` then interleave over 32 lanes. The reader is `divide(8,8)` then interleave of the `ldmatrix` atom over the four matrices. Staging is repeat on the storage, which exercises the swizzled-repeat rule, together with a stage axis on the thread maps, and slicing is restrict on the composites. Four properties are asserted address by address. Each lane’s global vector is contiguous and consecutive lanes in a quad read consecutive 16-byte vectors. Every shared-memory vector is 16-byte aligned and contiguous, so the swizzle does not break vectorization. The writer and reader thread–value maps are each bijective onto all 512 staged elements. And for every logical element, the byte the copy writes is the byte the mma fetch reads, across two pipelines derived independently. No composite in the suite is written by hand.

9 Related work

CuTe/CUTLASS [1, 2] is the system under study; its operations and their conditions are summarized in Section 1. **Shah** [3] formalizes the algebra and gives the admissibility conditions. They answer when two layouts fuse into one, which is a question this algebra never asks. **Carlisle et al.** [4] give CuTe’s algebra categorical foundations. **Triton’s linear layouts** [5] model layouts as $\mathbb{F}_2$ -linear maps on address bits, gaining generic layout conversion; that work is about what a representation can express, where ours is about which maps may be built, and the two combine: an $\mathbb{F}_2$ matrix is one way to implement a swizzle here. **Bhaskaracharya et al.** [6] model both CuTe and Triton layouts as integer set relations and reason about composition, inversion, and complement with a general Presburger solver; the question of Section 4.2 is expressible there, and the point of our procedure is that on a finite box it needs no solver—one pass of first differences per axis. **Axe** [7] unifies tiling, sharding, replication, and offsets over named axes, from device meshes down to the lane, warp, and register mappings of tensor-core tiles—the same grain as ours, over a wider span—and makes replication explicit as we do. Its Appendix A.2.1 contains the characterization Section 4.2 builds on, with the attribution and the difference stated there. **Halide** [9], **TVM** [10], and **MLIR** [11] carry bounds-aware integer expression simplifiers, which is the tool one reaches for first here, and Section 4.1 reports why a decision procedure replaced ours.

Taken one at a time the distinctions this design rests on are familiar. Replication is explicit in Triton and in Axe, the read and write asymmetry is known to every kernel author, and slicing a view without rewriting it is ordinary practice. What is new is that all three are enforced together: replication is a constructor that may be read and not stored through, nothing maps out of an address, and a restriction sits beside an unmodified layout and so applies to composites. Those requirements, rather than any condition about fusing, are what make a composition legal.

10 Scope and limitations

The domain is dense tiled tensor-core kernels, the workload CuTe exists for, and every claim here is scoped to it. Ragged tiles are handled by masking, which is outside the algebra. Negative strides are unsupported. Tilers for divide that are layouts rather than shapes are deferred until a kernel needs them. Density is decided by Lemma 1 without enumeration, and the test suite checks that criterion against an enumeration oracle on 400 random cases; alias-freedom, injectivity, and anything about a composite cost $O(\text{size})$ enumeration when the kernel is compiled, and the decision procedure of Section 4.2 costs a further $O(\text{size})$ evaluations. Deciding that question with extents known only at run time is outside this scope, where tile shapes are compile-time constants.

11 The language this serves

This algebra describes data movement for a warp-level GPU language we are building, in which the unit of execution in the source is the warp, lanes are SIMD data and divergence cannot be written, warps communicate through pipes that compile to barrier machinery, and every buffer through a pipe carries a layout from this paper. The language is future work and the algebra stands without it.

References

- [1] NVIDIA. *CuTe: Layouts and Layout Algebra*. CUTLASS documentation, docs.nvidia.com/cutlass, accessed 2026.
- [2] C. Cecka. *CuTe Layout Representation and Algebra*. NVIDIA Research; arXiv:2603.02298, 2026.
- [3] J. Shah. *A Note on the Algebra of CuTe Layouts*. Colfax Research technical note, 2024.
- [4] J. Carlisle, J. Shah, R. Stern, P. VanKoughnett. *Categorical Foundations for CuTe Layouts*. arXiv:2601.05972, 2026.
- [5] K. Zhou, M. Lezcano, A. Goucher, A. Rakhmati, J. Niu, J. Lebar, P. Szczerbuk, P. Bell, P. Tillet, T. Raoux, Z. Moudallal. *Linear Layouts: Robust Code Generation of Efficient Tensor Computation Using $\mathbb{F}_2$* . ASPLOS 2026; arXiv:2505.23819.
- [6] S. G. Bhaskaracharya, A. Acharya, B. Hagedorn, V. Grover. *Modeling Layout Abstractions Using Integer Set Relations*. arXiv:2511.10374, 2025.
- [7] B. Hou, H. Jin, G. Wang, J. Chen, Y. Cai, L. Yang, Z. Ye, Y. Ding, R. Lai, T. Chen. *Axe: A Simple Unified Layout Abstraction for Machine Learning Compilers*. arXiv:2601.19092, 2026.
- [8] NVIDIA. *Parallel Thread Execution ISA*. Matrix fragment layout tables, accessed 2026.
- [9] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, S. Amarasinghe. *Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines*. PLDI 2013.

- [10] T. Chen et al. *TVM: An Automated End-to-End Optimizing Compiler for Deep Learning*. OSDI 2018.
- [11] C. Lattner et al. *MLIR: Scaling Compiler Infrastructure for Domain Specific Computation*. CGO 2021.